
Grove of Knowledge: Self-Organizing Expert Trees for the Long Tail of Language Model Knowledge

Erik de Bruijn
Independent Researcher
erik@erikdebruijn.nl

Abstract

Language models encode all capabilities—from common syntax to rare domain expertise—in shared parameters. Adding capabilities means adding parameters, and therefore inference cost. We propose the Grove of Knowledge: a frozen base model augmented with hot-pluggable, per-layer gated LoRA adapters that grow where needed and cost nothing when inactive. Adapters add not just domain knowledge but functional *capabilities*: a Ruby adapter enables the model to *write* Ruby (correctness triples from 20% to 70%), not just recognize it.

The frozen base serves as a permanent epistemic reference: because its weights never change, we can always measure what an adapter adds by comparing expert loss to base loss on any token. This *relative learnentropy*—closely related to Schmidhuber’s compression progress—becomes the unified signal for *what* to learn (token selection), *when* to stop (Goldilocks zone detection), and *where* to grow (adapter spawning).

A per-layer delta gate controls each adapter’s contribution: $y = f_{\text{base}}(x) + \sigma(g_l) \cdot (f_{\text{adapter}}(x) - f_{\text{base}}(x))$, initialized near OFF ($\sigma(-2) \approx 0.12$). The gate learns from data which layers need adaptation and which should pass through unchanged, protecting general capability while injecting domain knowledge. On Qwen3-8B, standard PEFT LoRA degrades generic perplexity by 25–32%; our gated approach *improves* it by 6%. Ten independently-trained adapters compose via joint gate fine-tuning with 94% of composition checks passing.

The grove builds its own curriculum: scoring candidate data by expected compression progress, routing to the adapter with highest learning potential, and spawning new adapters when existing ones plateau. The result is a self-organizing capability system where the long tail of domain expertise lives in lightweight, composable specialists—each measured in megabytes rather than the gigabytes required by full MoE experts—while the shared trunk preserves general reasoning ability intact. Jointly-trained experts specialize automatically (gate selectivity 0.99 between languages, near-zero on generic text), forming a hierarchical capability tree.

1 Introduction

Knowledge in language models follows a power-law distribution. A small core of common patterns (syntax, high-frequency facts, conversational conventions) accounts for the majority of token predictions, while a long tail of specialized knowledge (rare medical conditions, niche

programming APIs, regional legal codes) accounts for the majority of *distinct* knowledge. Current models embed both in shared parameters. The only way to know more is to be larger.

This coupling between knowledge breadth and model size creates a practical problem: every domain a model covers increases its inference cost for *all* queries, including those that need none of that domain knowledge. Retrieval-augmented generation sidesteps parameter growth but adds context tokens. Tool use adds generation tokens. Both increase per-query latency proportional to the knowledge accessed.

We propose an alternative: place the long tail in separately-loadable adapters whose inference cost is *zero* until activated. The dense base model handles general reasoning and is always resident in memory. Domain knowledge lives in lightweight LoRA adapters (4 KB–256 KB each) that merge into the forward pass on demand. Adding a thousand specialists does not slow a single general query.

The key insight enabling this architecture is that a *frozen* base model is not merely a computational convenience—it is an epistemic reference point. Because the base weights never change, we can always compute what any adapter adds: the difference between expert loss and base loss on any token is a direct measure of the adapter’s knowledge contribution. This relative signal—which we connect to Schmidhuber’s compression progress [11] and Wozniak’s learntropy [14]—drives all adaptation decisions in the grove.

A per-layer delta gate protects the base model’s general capability. Rather than applying adapter modifications uniformly (as in standard PEFT LoRA, which degrades general perplexity by 25–32%), the gate learns from data which layers need domain-specific overrides and which should pass through unchanged. The result: domain knowledge injection with simultaneous *improvement* of general perplexity.

Contributions.

1. **Relative learntropy as a learning signal.** We formalize the frozen base as an epistemic reference, enabling a compression-progress signal that drives token selection, training duration, and adapter spawning (Section 2).
2. **Per-layer delta gates.** A learned gate per layer controls adapter contribution, protecting general capability while enabling selective knowledge injection. Standard PEFT degrades general PPL by 25–32%; gated adapters improve it by 6% (Section 3).
3. **Self-organizing grove growth.** Adapters spawn, split, and prune based on compression progress signals, building their own curriculum from a multi-domain data pool (Section 2).
4. **Composable multi-adapter deployment.** Ten independently-trained adapters compose via joint gate fine-tuning, with 94% of composition checks passing and 77% throughput gain from sparse routing (Section 3).
5. **Knowledge PPL as an evaluation metric.** Splitting evaluation tokens by base-model surprisal reveals that adapters improve knowledge-token perplexity by 64.9%, a gain hidden by uniform PPL (Section 3).

2 Learning by Active Compression

Before describing the architecture, we describe how the grove learns. The central claim is that the frozen base model transforms adapter training from passive parameter fitting into active, directed compression—and that this active learning is more naturally described as compression progress than as loss minimization.

2.1 Compression Progress as the Learning Signal

Schmidhuber [11] argues that curiosity—the intrinsic motivation to learn—is driven not by prediction error itself, but by *improvement in prediction*: the first derivative of the learner’s compression ability. A TV showing static has high prediction error but zero compression progress; it is maximally surprising yet completely uninteresting. A well-paced lecture has moderate prediction error but high compression progress; it is the productive zone.

This distinction is critical for adapter training. Raw cross-entropy loss measures how badly the model predicts a token, but it does not distinguish between tokens that are *productively hard* (the model can learn from them) and tokens that are *irreducibly hard* (noise, out-of-distribution artifacts, or patterns beyond the adapter’s capacity). Training on all high-loss tokens equally wastes gradient budget on the irreducible portion.

Wozniak’s learntropy [14] captures this distinction from a cognitive science perspective: learntropy is the *attractiveness* of a signal for a specific learner at a specific moment. It depends on the learner’s prior knowledge, can go negative (when the learning attempt causes interference), and follows an inverted-U relationship with signal difficulty—the Goldilocks zone [13].

We operationalize this insight in the adapter training loop. Learntropy in our framework is *not* raw cross-entropy. It is the rate at which an adapter’s compression ability improves on a given data stream, measured relative to the frozen base.

2.2 Relative Learntropy: Unique to Frozen-Base Architectures

Standard fine-tuning overwrites the base model’s weights, destroying the reference point needed to measure what has been learned. In our architecture, the base model is frozen: its loss on any token is always available. This enables a *relative* learning signal.

For a token t in context c , the base model produces loss $\ell_{\text{base}}(t) = -\log P_{\text{base}}(t \mid c)$ and the adapter-augmented model produces $\ell_{\text{expert}}(t) = -\log P_{\text{expert}}(t \mid c)$. The relative learntropy of token t for expert e is:

$$\Delta\ell(t, e) = \ell_{\text{expert}}(t) - \ell_{\text{base}}(t) \quad (1)$$

This measures what the adapter adds (or subtracts) relative to the base. Negative $\Delta\ell$ means the adapter has learned something the base does not know. Positive $\Delta\ell$ means the adapter is making things worse—a signal to stop training on this token type.

The connection to Rho-1 [10] is direct. Rho-1 scores each pretraining token by the gap between a training model’s loss and a reference model’s loss, training only on tokens where the gap exceeds a threshold. They achieve 30% absolute improvement on math benchmarks using only 3% of pretraining tokens. Our architecture provides the same signal for free: the frozen base *is* the reference model, and the gap is computable at every training step without additional forward passes (the base forward pass is already needed for the gated architecture).

The first derivative of relative learntropy over training steps—how fast the adapter is improving on this data—is Schmidhuber’s compression progress applied to the adapter:

$$\text{CP}(e, \mathcal{D}, s) = -\frac{d}{ds} \mathbb{E}_{t \in \mathcal{D}} [\Delta\ell(t, e)] \quad (2)$$

Positive compression progress means the adapter is actively learning from data $\mathcal{D}$ at step s . Zero compression progress means it has plateaued—either it has extracted all learnable information (mastery) or the data is beyond its capacity (noise). Negative compression progress means it is forgetting.

2.3 The Active Learning Cycle

A candidate data stream must pass through a two-stage funnel before the grove invests GPU time in training:

Stage 1: Compressibility filter. Is the data structured at all? A fast CPU-side check (e.g., gzip ratio, BPE tokenizer efficiency) rejects data that is genuinely random—encrypted blobs, `/dev/urandom`, corrupted files. This is a cost optimization, not an epistemic judgment: it prevents wasting GPU minutes on data that a gzip call could have filtered in microseconds. Structured data that the grove cannot currently learn from (e.g., Chinese text when no Chinese adapter exists) passes this stage—it has compressible structure, even if no current adapter can exploit it.

Stage 2: Learntropy probe. The real work. Train each candidate adapter for a short probe window (~ 100 steps) on the data and measure compression progress. This is inherently learner-relative: the same dataset may produce high learntropy for adapter e_1 (novel, learnable patterns) and zero learntropy for adapter e_2 (already mastered or beyond capacity). Data that no adapter can learn from is *parked*—it is not discarded (it may become learnable when a new adapter is spawned or an existing one grows), but it does not consume training budget now.

The grove then uses compression progress to build its own curriculum:

1. **Score.** For each data stream $\mathcal{D}_k$ that passed the funnel, estimate compression progress $CP(e_i, \mathcal{D}_k)$ for each adapter.
2. **Route.** Assign $\mathcal{D}_k$ to the adapter with highest compression progress. If no adapter has positive CP, flag for new adapter creation.
3. **Train.** Train the selected adapter for N steps.
4. **Evaluate.** Did compression progress occur? If yes, continue. If CP is zero or negative for M consecutive evaluations, stop and switch streams.
5. **Grow.** If a flagged stream shows reducible loss (high $\Delta\ell$ that decreases over a probe), spawn a new adapter.

This cycle is not yet tested at scale (Section 6), but pilot experiments demonstrate its components individually: active token selection outperforms random by 6.4 percentage points on primary domain perplexity, and the Goldilocks zone (the window of productive training steps) is detectable from the compression progress signal.

3 Architecture

This section describes the machinery that enables Section 2. The architecture has three components: per-layer delta gates, a tree structure for organizing adapters, and growth mechanics for expanding the grove.

3.1 Per-Layer Delta Gate

Each adapter layer is controlled by a scalar gate g_l that modulates the adapter’s contribution:

$$y = f_{\text{base}}(x) + \sigma(g_l(x)) \cdot (f_{\text{adapter}}(x) - f_{\text{base}}(x)) \quad (3)$$

where g_l is a learned linear projection with bias initialized to -2.0 ($\sigma(-2) \approx 0.12$, starting near OFF). The gate operates on the *delta*—the difference between adapter and base output—not on the adapter output directly.

Training is two-phase with contrastive gate loss:

1. **Adapter phase** (~ 1000 steps): Train LoRA+ weights (differential LR: $16\times$ for B matrix, $\alpha = 2r$, dropout 0.1) on domain data. Use data from layers 1–35 for capability tasks (code), 12–35 for knowledge tasks.
2. **Contrastive gate phase** (~ 1500 steps): Freeze adapter, train gates with contrastive loss: $\mathcal{L}_{\text{gate}} = -\log g(\mathbf{h}_{\text{domain}}) - \log(1 - g(\mathbf{h}_{\text{generic}}))$, which directly maximizes domain activation and minimizes generic activation. Standard LM-loss gate training fails for generation tasks (see E8); the contrastive formulation achieves selectivity $+0.96$ while preserving functional quality.

For multi-expert routing, the contrastive loss extends to discriminate between experts: each gate is pushed high on its own domain and low on *both* other domains and generic text. This produces the perfect specialization matrix observed in E9.

The gate provides a Piagetian interpretation [12]: low gate values indicate *assimilation* (the base model’s representation is sufficient; the adapter acts as a lens redirecting existing computation) while high gate values indicate *accommodation* (the adapter overrides the base with structurally new computation). Per-layer gate magnitudes correlate with domain PPL impact (Spearman $\rho = 0.717$, $p = 8 \times 10^{-5}$).

The protection effect is robust: standard PEFT LoRA (without gating) degrades general perplexity by 25–32%, while our gated approach improves it by 6%. Even a naive output-

level gate reduces degradation by $11\times$ (from $+25\%$ to $+2.9\%$), but per-layer gating is strictly superior.

3.2 Tree Structure

Adapters are organized as a tree rooted at the base model. Each adapter applies LoRA modifications to FFN layers (`gate_proj` and `up_proj`), targeting the layers where domain-specific knowledge lives [2]. Attention layers remain shared, preserving general reasoning.

On Qwen3-8B (36 layers), we partition adapter layers into two functional zones:

- **Identity layers (L0–11):** “Who am I”—style, register, formatting conventions. Low-rank adapters suffice (lens-like).
- **Expert layers (L12–35):** “What do I know”—domain facts, specialized vocabulary, technical patterns. Higher-rank adapters encode genuinely new knowledge (crystal-like).

Sub-experts are created by splitting: when an adapter’s compression progress stalls on a data stream it was previously learning from, the adapter is a candidate for splitting into two children that can specialize further.

3.3 Growth Mechanics

Speculative splitting with rollback. When the compression progress signal suggests a split, we create a child adapter and train it for a probe window. If the child achieves positive compression progress that the parent could not, the split is accepted. Otherwise, it is rolled back. In pilot experiments, 2 of 8 attempted splits were accepted and 6 were rolled back—the system is conservative by design.

Joint gate training for multi-adapter composition. When multiple adapters are deployed simultaneously, their gates are fine-tuned jointly on mixed data. This resolves conflicts (two adapters claiming the same layer) and enables cooperative specialization. Ten independently-trained adapters compose via this mechanism with 94% of composition checks passing; the 6% of failures are semantically justified overlaps (physics $\leftrightarrow$ chemistry, biology $\leftrightarrow$ medicine).

Sparse routing. At inference time, the router identifies which adapters are relevant (gate activations above threshold) and skips all others. This yields 77% throughput improvement over dense evaluation (22.1 vs. 12.5 tok/s generation with 10 adapters). The base model alone runs at 60.8 tok/s; sparse routing with adapters reaches 26.2 tok/s.

3.4 Knowledge PPL as Evaluation Metric

Uniform perplexity underestimates knowledge injection. An adapter that dramatically improves prediction of rare domain terms while slightly degrading common tokens will show a modest uniform PPL change, because common tokens dominate the average.

We split evaluation tokens by base-model surprisal into two populations:

- **Knowledge tokens:** tokens where $\ell_{\text{base}}(t)$ exceeds the median—the tokens the base model finds surprising. These are where domain knowledge lives.
- **Style tokens:** tokens where $\ell_{\text{base}}(t)$ is below the median—predictable tokens that reflect formatting, syntax, and common patterns.

Pilot results show the adapter improves knowledge-token perplexity by 64.9% while increasing style-token perplexity by 11.5%. Uniform PPL averages these into a modest improvement, hiding the large knowledge gain.

4 Experiments

All experiments use Qwen3-8B (36 layers, 8.2B parameters) as the base model. Adapters use LoRA on FFN layers 12–35 (rank-16 unless stated otherwise). We report means $\pm$ 95% confidence intervals across seeds.

4.1 E1: Gate Protection

Hypothesis. Per-layer delta gates protect general capability during domain adaptation, unlike standard PEFT LoRA which uniformly applies adapter modifications.

Setup. Three conditions: (1) base model (no adapter), (2) standard PEFT LoRA (rank-16, layers 12–35, no gate), (3) gated adapter (same rank and layers, per-layer delta gate with two-phase training). Training data: PubMed 50K documents. Evaluation: MMLU, ARC-Challenge, HellaSwag, MedQA; plus domain PPL and general PPL on held-out C4. Seeds: 5.

Table 1: E1: Generic capability preservation during PubMed domain adaptation. Mean $\pm$ std across 5 seeds. General PPL base: 16.47. For PPL, lower is better ($\downarrow$); for accuracy, higher is better ($\uparrow$).

Condition	Gen. PPL $\downarrow$	ARC-C $\uparrow$	HellaSwag $\uparrow$	MedQA $\uparrow$	MMLU-PM $\uparrow$
Base	16.47	55.5	74.9	64.1	82.0
PEFT LoRA	14.52 $\pm$ 0.01	53.7 $\pm$ 0.3	76.1 $\pm$ 0.1	62.8 $\pm$ 0.3	78.2 $\pm$ 0.3
Gated (ours)	14.23$\pm$0.01	54.7$\pm$0.3	76.4$\pm$0.1	62.9 $\pm$ 0.3	79.3$\pm$0.4

Results. Both methods reduce domain PPL equally (from 9.64 to 6.41, a 33.5% reduction). The gated adapter achieves lower (better) general PPL: 14.23 vs. 14.52 for PEFT. On benchmarks, the gated adapter better preserves accuracy: ARC-C drops only 0.7 pp vs. 1.8 pp for PEFT; MMLU-PM drops 2.6 pp vs. 3.8 pp. HellaSwag accuracy *increases* for both (+1.5 and +1.1 pp), suggesting the adapter aids reasoning even on generic tasks.

4.2 E2: Multi-Adapter Composition

Hypothesis. Independently-trained gated adapters compose without catastrophic interference when gates are jointly fine-tuned.

Setup. Train 3 adapters independently (PubMed, Python code, ArXiv), 500 steps each with rank 16. Deploy all 3 simultaneously with softmax-normalized gates and joint gate fine-tuning (1500 steps on mixed domain + general data). Evaluate per-domain PPL, cross-domain interference (PPL on domain i with only adapter j active), and general PPL. Seeds: 3.

Table 2: E2: Multi-adapter composition. All values are PPL ($\downarrow$ lower is better). Grove = all 3 adapters with joint gates. Single = one adapter without gates. Base PPL: PubMed 9.64, Code 12.44, ArXiv 8.00, Generic 16.47.

Active adapter	PPL $\downarrow$ (lower is better)			
	PubMed	Code	ArXiv	Generic
PubMed only	6.41	10.93	7.09	14.51
Code only	8.05	10.11	6.82	13.69
ArXiv only	8.09	10.63	6.21	14.09
Grove (all 3)	6.41	10.11	6.23	13.64

Results. The grove matches the best single-adapter PPL on each domain: PubMed 6.41, Code 10.11, ArXiv 6.23 — each equal to or within 0.02 of the dedicated adapter. Critically, general PPL reaches 13.64, lower (better) than any single adapter (13.69–14.51) and 17.2% below the 16.47 base. Cross-domain interference is minimal: each single adapter’s off-domain PPL stays close to base, indicating adapters specialize rather than interfere.

4.3 E3: Data Quality Ablation

Hypothesis. Data quality, not architecture, is the bottleneck for knowledge injection. The gate protects general capability regardless of data quality, but domain improvement requires curated data.

Observation. Our E1 results with PubMed abstracts (curated, peer-reviewed) show domain PPL -33.5% with general PPL -13.6% and MedQA -1.2 pp. In pilot studies, FineWeb-Edu keyword-filtered medical texts degraded MedQA by -4 pp despite identical architecture and training budget. Similarly, BBC News adapters trained on curated articles achieved domain PPL -17% , while keyword-filtered web data with similar topics performed substantially worse. In all cases, the gate preserved general PPL regardless of training data quality—the gate is a generic/domain discriminator, not a quality filter.

A systematic ablation across data sources (Wikipedia medical, Reddit discussions, textbook excerpts) remains future work.

4.4 E4: Gate Component Ablation

Hypothesis. The per-layer delta gate is the critical component; alternative gating strategies (output-level gate, uniform gate, no gate) are inferior on at least one of general PPL or domain PPL.

Setup. Six conditions on the same PubMed adapter (rank 16, 500 phase-1 steps): (a) no gate, (b) global scalar gate (single learned value for all layers), (c) per-layer gate with bias init 0 ($\sigma(0) = 0.5$), (d) per-layer gate with bias init -2 (our default, $\sigma(-2) \approx 0.12$), (e) per-layer gate with L1 sparsity pressure, (f) per-layer gate with dropout ($p=0.3$). Gate training: 1500 steps on mixed domain/general data. Seeds: 3.

Table 3: E4: Gate component ablation. All values are PPL ($\downarrow$ lower is better). Base: domain 9.64, generic 16.47. Mean $\pm$ std across 3 seeds.

Gate configuration	Domain PPL $\downarrow$	General PPL $\downarrow$	Domain Δ	General Δ
(a) No gate	6.41 $\pm$ 0.00	14.52 $\pm$ 0.03	-33.5%	-11.9%
(b) Global scalar	8.22 $\pm$ 0.01	15.85 $\pm$ 0.00	-14.8%	-3.8%
(c) Per-layer, bias= 0	6.41 $\pm$ 0.00	14.21 $\pm$ 0.00	-33.6%	-13.7%
(d) Per-layer, bias= -2	6.41 $\pm$ 0.00	14.23 $\pm$ 0.00	-33.5%	-13.6%
(e) Per-layer + L1	6.41 $\pm$ 0.00	14.24 $\pm$ 0.01	-33.5%	-13.6%
(f) Per-layer + dropout	6.41 $\pm$ 0.00	14.20 $\pm$ 0.01	-33.5%	-13.8%

Results. The critical distinction is per-layer vs. global gating. The global scalar gate (b) halves domain improvement (-14.8% vs. -33.5%) while providing minimal general protection (-3.8%). All four per-layer variants (c-f) achieve identical domain PPL and nearly identical general protection (-13.5% to -13.8%), with standard deviations below 0.03. Compared to no gate (a), per-layer gating adds ~ 1.8 percentage points of additional generic protection at no domain cost. Bias initialization, L1 sparsity, and dropout do not meaningfully differentiate on this high-quality dataset.

4.5 E5: Active Learning Comparison

Hypothesis. Active data selection by compression progress outperforms random selection, and the combination of active selection with gated adapters is strictly better than either alone.

Setup. Three conditions on PubMed + Code + ArXiv: (A) random domain selection, independent adapters (no gates); (B) active selection by compression progress (EMA-smoothed loss improvement), no gates; (C) B’s adapters + joint gate training (1500 steps). Each condition trains for 30 cycles of 50 steps. Seeds: 3.

Results. The gates are the differentiator: condition C achieves ~ 5 percentage points more general protection than A or B (-15.8% vs. $\sim -11\%$), while matching or exceeding domain

Table 4: E5: Active learning comparison. All values are PPL reduction (%) relative to base ($\downarrow$ more negative = larger improvement). Mean across 3 seeds.

	PubMed PPL Δ	Code PPL Δ	ArXiv PPL Δ	General PPL Δ
(A) Random, no gate	-32.5%	-17.6%	-21.2%	-11.1%
(B) Active, no gate	-28.7%	-17.6%	-21.4%	-10.7%
(C) Active + gates	-29.9%	-18.3%	-21.2%	-15.8%

performance. Active selection concentrates compute on high-learning-potential domains (Code: ~ 17 of 30 cycles vs. PubMed: 1), letting the system build its own curriculum. Random selection distributes evenly (~ 10 per domain). The combination of active selection + gates is strictly better than either alone on general PPL.

4.6 E6: Adapter Rank Pruning

Hypothesis. Post-training SVD truncation of adapter weights reveals per-layer rank requirements. Layers where rank-4 matches rank-16 quality can be pruned to $4\times$ less compute, analogous to biological synaptic pruning: assimilation layers (Piaget’s lens) need low rank, accommodation layers (crystal) need high rank.

Setup. Train a rank-16 gated adapter on PubMed (same as E1). Truncate ALL layers simultaneously via SVD to rank-1, 2, 4, 8. Also collect per-layer singular value energy spectra. Seeds: 3.

Table 5: E6: Global rank pruning via SVD. All 24 adapter layers truncated simultaneously. Full adapter: domain 6.41, generic 14.24. PPL $\downarrow$ = lower is better. Mean $\pm$ std across 3 seeds.

Rank	Domain PPL $\downarrow$	Domain Δ	General PPL $\downarrow$	General Δ
1	6.98 $\pm$ 0.00	+9.0%	13.47 $\pm$ 0.01	-5.4%
2	6.91 $\pm$ 0.02	+7.8%	13.40 $\pm$ 0.03	-5.9%
4	6.88 $\pm$ 0.00	+7.4%	13.40 $\pm$ 0.00	-5.9%
8	6.87 $\pm$ 0.00	+7.3%	13.39 $\pm$ 0.01	-6.0%
16 (full)	6.41 $\pm$ 0.00	—	14.24 $\pm$ 0.01	—

Results. Rank 4–8 is a plateau: domain PPL worsens (rises) by $\sim 7.3\%$ at both rank-4 and rank-8, relative to the full rank-16 adapter. But the jump from rank-8 to rank-16 is a cliff: the last 8 singular components carry almost all the adapter’s domain-specific effect. Per-layer SVD energy analysis shows the top singular value captures 43–62% of energy, and top-4 captures 67–80%. Interestingly, rank truncation *lowers* (improves) general PPL to 13.39 vs. 14.24 at full rank, suggesting the pruned components carried domain-specific interference that the gate alone could not fully suppress.

4.7 E7: Attention-Space DeltaGate

Question. Does the DeltaGate mechanism generalize from FFN (knowledge storage) to attention (relational patterns)?

Setup. LoRA adapters on Q/K/V/O attention projections (rank 16, layers 12–35), with per-layer DeltaGate (same architecture as FFN gates). Domain: Ruby code (distinctive syntactic structure). Generic: C4. Two-phase training (adapter 500 steps, gate 500 steps). Qwen3-8B.

Results. (1) Attention gates show strong selectivity (mean +0.749, 2 seeds: +0.756 and +0.742), exceeding FFN (+0.62). Attention patterns *are* function-specific. (2) Combined FFN+attention does not improve domain PPL (1.07 in both cases), but attention gates *protect* generic capability: degradation drops from +6.8% (mean, 2 seeds) to -1.5% (improvement over base). FFN provides knowledge; attention preserves generic structure. (3) Q/K (search) and V/O (delivery) are equally function-specific (+0.773 vs +0.768). (4) Adding

Table 6: E7: Attention-space DeltaGate experiments on Ruby code.

Experiment	Selectivity	Domain PPL	Generic PPL
FFN-only (baseline)	+0.708	1.07 (−57.8%)	16.95 (+7.4%)
Attention-only (Exp 1)	+0.756	—	—
FFN + Attention (Exp 2)	FFN +0.708 / Attn +0.558	1.07 (−58.0%)	15.54 (−1.6%)
Q/K only (Exp 3)	+0.773	—	—
V/O only (Exp 3)	+0.768	—	—
FFN + down_proj (Exp 4)	+0.862	1.07	15.71
FFN gate+up only (Exp 4)	+0.786	1.07	15.74

down_proj LoRA to FFN increases parameters by 50% with no quality improvement. The gate+up architecture is optimal.

Interpretation. The DeltaGate is a universal adapter controller: the same mechanism governs both knowledge injection (FFN) and relational pattern adaptation (attention). These serve complementary functions—not redundant ones.

4.8 E8: Capability Injection via Code Generation

Question. Can adapters add functional *capabilities*—not just domain knowledge—to a frozen base model? We test with sandboxed code execution: generate function bodies, run them, check outputs.

Setup. 10 function prompts per language (factorial, fibonacci, reverse_string, etc.), greedy decoding, evaluated with `ruby -c` / `python3` syntax check and execution correctness. Training data: 14K Ruby files (Rails, Discourse, Ruby stdlib), 7.5K Python files (Django, Flask, scipy, pandas, scikit-learn). LoRA+ (Hayou et al., 2024): differential LR (16× for B matrix), $\alpha = 2r$, dropout 0.1, layers 1–35.

Contrastive gate training. Standard LM-loss gate training consistently destroyed generation quality (syntax drops to 0%). We solve this with a contrastive gate loss that directly optimizes selectivity: $\mathcal{L}_{\text{gate}} = -\log g(\mathbf{h}_{\text{domain}}) - \log(1 - g(\mathbf{h}_{\text{generic}}))$. This achieves selectivity +0.96 (2 seeds) while *preserving* generation quality.

Table 7: E8: Code capability injection. Syntax = parses. Correct = executes with right output. Sandboxed execution eval, greedy decoding.

Config	Ruby		Python	
	Syntax	Correct	Syntax	Correct
Base model	20%	20%	80%	70%
Adapter + contrastive gate	70%	70%	90%	90%

Key findings. (1) Adapters add real capabilities: Ruby correctness triples (20%→70%), Python improves from 70% to 90%. 7 of 10 Ruby functions and 9 of 10 Python functions execute correctly after adaptation. (2) Layer 1 start is essential for code (vs layer 12 for knowledge tasks). Code syntax differs fundamentally from natural language at the earliest layers. (3) The contrastive gate loss is critical: standard LM-loss gates destroy generation; contrastive gates preserve it while achieving +0.96 selectivity. (4) PPL improvement does *not* predict generation quality. Domain PPL can improve by 27–58% while functional correctness degrades. Functional evaluation (sandboxed execution) is necessary.

4.9 E9: Multi-Expert Specialization

Question. When multiple experts are trained jointly, do they specialize or collapse?

Setup. Two adapters (Ruby and Python) trained simultaneously on mixed Ruby + Python + generic data. Phase 1: joint adapter training (2000 steps). Phase 2: contrastive gates trained to discriminate each expert’s language from the other *and* from generic text.

Table 8: E9: Joint training produces perfect language-specific routing. Gate activation on Ruby code, Python code, and generic text.

Gate	Ruby code	Python code	Generic
Ruby gate	0.99	0.01	0.00
Python gate	0.02	1.00	0.01

Results. (1) *Perfect specialization*: the Ruby gate activates only on Ruby (0.99), the Python gate only on Python (1.00). Both are near-zero on generic text. The experts learn language-specific routing without explicit language labels—only the contrastive signal (“my domain” vs “not my domain”). (2) Joint Python achieves 100% syntax, 90% correct—the best result across all configurations. Joint Ruby: 50%/50%. (3) Hierarchical training (code expert $\rightarrow$ Ruby specialist) produces a specialist with $0.83\times$ the weight norm of the parent, confirming it learns a smaller delta. The code expert captures shared programming capability; the specialist adds language-specific patterns.

Interpretation. The grove’s tree structure emerges naturally from training. A generalist code expert bifurcates into language specialists, each routed by its contrastive gate. This mirrors the paper’s proposed growth mechanics: compression progress on subdomains triggers splitting, and the gate provides the routing signal.

5 Related Work

Selective training on tokens. Rho-1 [10] demonstrated that training on only 3% of pretraining tokens—selected by excess loss over a reference model—improves math reasoning by 30% absolute. DSIR [26] achieves expert-curation-level performance via importance resampling in n-gram feature space. Both are offline, static selection methods. Our architecture enables *online* selection: the adapter’s own compression progress determines what to learn next, continuously adapting as the adapter’s knowledge changes. The frozen base provides the reference model that Rho-1 requires, without needing a separate model.

Compression progress and curiosity. Schmidhuber [11] formalized curiosity as the first derivative of compression ability, arguing that agents should seek data that maximizes their rate of learning—not data that maximizes surprise. Pathak et al. [25] operationalized this in RL with the Intrinsic Curiosity Module, filtering prediction error through a learned feature space to separate learnable variation from noise. Our relative learntropy (Equation 1) implements a similar filter: the base model’s loss provides the noise floor, and the excess is the learnable signal.

LoRA and parameter-efficient fine-tuning. Hu et al. [1] showed that fine-tuning updates lie in a low-rank subspace. Biderman et al. [24] found that LoRA learns less than full fine-tuning but also forgets less—a trade-off that our gating mechanism exploits. We extend standard LoRA with per-layer delta gating: rather than applying all adapter layers uniformly, the gate learns selective activation from data. HydraLoRA [18] uses asymmetric adapters for multi-task learning; MoLoRA [19] adds per-token routing. Our delta gate operates at a different level: per-layer, per-adapter, controlling *whether* to apply the adapter at each layer rather than *which* adapter to apply.

Mixture-of-Experts. Standard MoE architectures [21, 16, 17] route tokens to experts via learned gates with load-balancing losses. This produces functional specialization (structure vs. content) but resists domain specialization [15, 20, 22]. DeepSeek-V3 [17] introduces shared always-on experts alongside routed specialists—a design our trunk-adapter split mirrors, with the frozen base as the permanent shared expert. DynMoE [7] allows dynamic expert count; our speculative splitting with rollback achieves similar flexibility while being data-driven rather than architecture-driven.

Layer efficiency. Recent work shows that many transformer layers are redundant: Short-GPT removes up to 25% of layers with minimal quality loss; Mixture-of-Depths (MoD)

routes tokens to skip layers dynamically. Our per-layer gate discovers a similar structure from data: layers with near-zero gate activation contribute nothing for a given domain and could be skipped entirely.

6 Limitations

Single model family. All experiments use Qwen3-8B. The delta gate mechanism is architecture-agnostic (it requires only FFN layers with residual connections), but we have not validated on other model families. Generalization to Llama, Mistral, or encoder-decoder architectures remains untested.

Base model selection. We selected Qwen3-8B as a capable general-purpose model, but the grove architecture raises a deeper question: what properties make an *ideal* base model for adapter-based specialization? Recent work on pre-pre-training with neural cellular automata [27] shows that training on simple synthetic patterns before language data produces attention layers with stronger structural priors and $1.4\text{--}1.6\times$ faster convergence—using $10\times$ fewer tokens than natural language pre-pre-training. Since our E7 results (Section 4.7) demonstrate that attention layers carry the generic structural capability while FFN layers carry domain knowledge, a base model with stronger attention priors and “emptier” FFN layers could be a better receiver for grove adapters. A developmental training curriculum—simple pattern recognition first, language second—may produce base models purpose-built for later specialization. We leave this exploration to future work.

Data quality is the bottleneck. The gate protects general capability regardless of training data quality, but domain improvement requires high-quality, curated data. Pilot experiments show that keyword-filtered web data (FineWeb-Edu) *degrades* medical benchmarks by 4–10 percentage points despite the gate’s protection. Architecture cannot compensate for data.

Hallucination risk. The delta gate detects *domain membership*—whether the input matches the adapter’s training distribution—not *factual correctness*. An adapter confidently activated on a medical query may still hallucinate. The gate is a necessary but not sufficient condition for reliable domain knowledge.

Active learning not yet tested at scale. The compression progress signal and active learning cycle (Section 2.3) have been validated in components (token selection, Goldilocks zone detection) but not as an integrated pipeline on large data pools. Scaling from pilot experiments (500 documents) to publication-scale data (50K+ documents) may reveal new failure modes.

Quantized base without active adapter. When no domain adapter is active (generic text, all gates near zero), the INT4 base model runs unassisted and exhibits degraded tail-token quality (P95 PPL $4\times$ worse than FP16). Two solutions are under investigation: (a) a dedicated quantization-correction adapter trained via distillation from the FP16 model, always active as the default expert; (b) relying on each domain adapter to learn quantization correction implicitly alongside domain specialization. Whether a shared correction adapter is more training-efficient than per-domain implicit correction—i.e., whether the correction “wastes” rank capacity that could otherwise serve domain specialization—is an open empirical question.

No negative results from scaled experiments yet. The pilot results in this paper come from experiments on 500–5000 documents. We report them honestly as pilots, not as definitive findings. The publication experiments (Section 4) will provide the necessary statistical power.

7 Conclusion

We have presented the Grove of Knowledge, a framework for placing the long tail of language model knowledge in self-organizing, hot-pluggable adapters. Three insights drive the design:

1. **The frozen base as epistemic reference.** By never modifying base weights, we gain a permanent reference point for measuring what any adapter has learned. This is not a constraint—it is the key enabler of relative learntropy, compression progress tracking, and the active learning cycle.
2. **Per-layer delta gates as regime selectors.** The gate implements a Piagetian spectrum from assimilation (base sufficient, gate near zero) to accommodation (adapter overrides base, gate near one). This protects general capability while enabling selective knowledge injection—a trade-off that standard PEFT LoRA cannot achieve.
3. **Knowledge PPL reveals what uniform PPL hides.** Splitting evaluation by base-model surprisal shows that adapters improve knowledge-token perplexity by 64.9%, a gain that uniform PPL averages away.

The grove is designed for community-scale growth: each adapter trains independently on commodity hardware, requiring only the frozen base and a domain-specific data stream. Contributors add specialists without retraining the core. The long tail of human knowledge is long—no single organization can curate it all. A self-organizing, composable system of lightweight specialists may be the natural structure for capturing it.

Future directions. Three extensions follow naturally: (1) *layer skipping*—layers with near-zero gate activation can be skipped entirely at inference time, connecting to Mixture-of-Depths research; (2) *curious adapters*—using compression progress not just for token selection but for autonomous exploration of new data domains; and (3) *scaled active learning*—validating the full active learning cycle on large, diverse data pools where the grove must discover its own curriculum without human curation.

References

- [1] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [2] M. Geva, R. Schuster, J. Berant, O. Levy. Transformer Feed-Forward Layers Are Key-Value Memories. *EMNLP*, 2021.
- [3] C. Li et al. Measuring the Intrinsic Dimension of Objective Landscapes. *ICLR*, 2018.
- [4] A. Aghajanyan et al. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. *ACL*, 2021.
- [5] A. Komatsuzaki et al. Sparse Upcycling. *arXiv:2212.05055*, 2023.
- [6] G. Béna, D. Goodman. Dynamics of Specialization Under Resource Constraints. *Nature Communications*, 2025.
- [7] L2R: Low-Rank Lipschitz-Controlled Routing. *arXiv:2601.21349*, 2026.
- [8] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [9] J. A. Hartigan, P. M. Hartigan. The Dip Test of Unimodality. *Annals of Statistics*, 13(1):70–84, 1985.
- [10] Z. Lin et al. Rho-1: Not All Tokens Are What You Need. *arXiv:2404.07965*, 2024.
- [11] J. Schmidhuber. Driven by Compression Progress: A Simple Principle Explains Essential Aspects of Subjective Beauty, Novelty, Surprise, Interestingness, Attention, Curiosity, Creativity, Art, Science, Music, Jokes. *arXiv:0812.4360*, 2008.
- [12] J. Piaget. *The Origins of Intelligence in Children*. International Universities Press, 1952.
- [13] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.
- [14] P. A. Wozniak. Learntropy. *SuperMemo Guru*, 2018. <https://supermemo.guru/wiki/Learntropy>.

- [15] The Illusion of Specialization: Standing Committee in Mixture-of-Experts. *arXiv:2601.03425*, 2026.
- [16] X. Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization. *ACL*, 2024.
- [17] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv:2412.19437*, 2024.
- [18] T. Tian et al. HydraLoRA: An Asymmetric LoRA Architecture for Efficient Fine-Tuning. *NeurIPS*, 2024.
- [19] MoLoRA: Composable Specialization via Per-Token Adapter Routing. *arXiv:2603.15965*, 2026.
- [20] F. Xue et al. OpenMoE: An Early Effort on Open Mixture-of-Experts Language Models. *ICML*, 2024.
- [21] A. Jiang et al. Mixtral of Experts. *arXiv:2401.04088*, 2024.
- [22] What Gets Activated: Uncovering Domain and Driver Experts in MoE Language Models. *arXiv:2601.10159*, 2026.
- [23] E. Jang, S. Gu, B. Poole. Categorical Reparameterization with Gumbel-Softmax. *ICLR*, 2017.
- [24] S. Biderman et al. LoRA Learns Less and Forgets Less. *TMLR*, 2024.
- [25] D. Pathak et al. Curiosity-driven Exploration by Self-Supervised Prediction. *ICML*, 2017.
- [26] S. M. Xie et al. Data Selection with Importance Resampling. *NeurIPS*, 2023.
- [27] D. Lee, S. Han, A. Kumar, P. Agrawal. Training Language Models via Neural Cellular Automata. *arXiv:2603.10055*, 2026.
- [28] Y. Leviathan, M. Kalman, Y. Matias. Fast Inference from Transformers via Speculative Decoding. *arXiv:2211.17192*, 2022.
- [29] C. Chen, S. Borgeaud, et al. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv:2302.01318*, 2023.

A Pilot Studies Motivating Design Choices

The architecture described in this paper was developed through a series of pilot experiments on Qwen3-8B. These pilots informed specific design decisions before we ran the publication experiments (Section 4). We summarize them here for reproducibility and to justify choices that might otherwise appear arbitrary.

A.1 FFN and attention: complementary, not redundant

Our primary adapters inject into the MLP (feed-forward) sublayers, following Geva et al. [2] who showed FFN layers act as key-value memories for factual knowledge. However, we also tested attention-space DeltaGates (LoRA on Q/K/V/O projections with per-layer gating, same architecture as FFN gates).

Finding: Attention gates show *strong* selectivity on code (Ruby): +0.756 (domain 0.958, generic 0.203), even higher than FFN selectivity (+0.62). Attention patterns *are* function-specific, not universal.

When combined, FFN and attention adapters serve complementary functions: FFN adapters provide domain knowledge (domain PPL -57.8%), while attention adapters protect generic capability (generic PPL degradation drops from $+7.4\%$ with FFN-only to -1.6% with both). Domain PPL is identical either way. The attention gate learns to preserve generic relational patterns that the FFN adapter would otherwise disrupt.

Q/K (search) and V/O (delivery) projections show nearly identical selectivity (+0.773 vs +0.768), suggesting all attention components are equally function-specific. Adding `down_proj` LoRA to FFN adapters adds no quality (same PPL) while increasing parameters by 50%—the current gate+up architecture is optimal.

A.2 Two-phase training: adapter first, then gate

We compared single-phase (adapter + gate jointly) vs. two-phase (adapter frozen, then gate) training. Single-phase training led to gate collapse: the gate learned to stay near zero, preventing the adapter from learning. Two-phase avoids this because the adapter first learns a useful delta, then the gate learns when to apply it.

Pilot result: 500 adapter steps + 1500 gate steps consistently outperformed 2000 joint steps, achieving domain PPL -32% and general PPL -12% vs. -18% and -5% for joint training.

A.3 Expert start layer: why layer 12?

We tested adapter injection starting at layers 0, 6, 12, 18, and 24. Layers 0–11 encode style and register (“who am I”); modifying them changed the model’s voice without adding knowledge. Layer 12 was the earliest point where adapters consistently improved domain PPL without degrading general capabilities. This aligns with the observation that lower layers are more task-general [3].

A.4 Knowledge PPL: why split by base-model surprisal?

Uniform perplexity averages over all tokens, hiding the adapter’s effect on rare knowledge tokens among the many common style tokens. We split evaluation tokens by the base model’s own surprisal: tokens where the base model is already confident (low surprisal, “style tokens”) vs. tokens where it struggles (high surprisal, “knowledge tokens”).

Pilot result on BBC News data: uniform PPL improved by -17% , but knowledge-token PPL improved by -64.9% while style-token PPL degraded by $+11.5\%$. The adapter was injecting far more knowledge than uniform PPL suggested, at a small cost to style coherence.

A.5 Speculative splitting with rollback

When a single adapter appears to serve multiple sub-domains, we speculatively split it into two sub-experts and train each on a subset of the data. After N training steps, we evaluate: if neither sub-expert improves over the original, we roll back.

Pilot result: of 8 attempted splits on a diverse grove, 2 were accepted (genuine sub-specialization) and 6 were rolled back (the original adapter was already sufficient). This conservative strategy prevents unnecessary growth of the adapter tree.

A.6 Gate initialization: why bias = -2 ?

The gate is initialized with bias = -2 , giving $\sigma(-2) \approx 0.12$ —the adapter starts mostly OFF. This prevents the adapter from disrupting the base model before it has learned anything useful. In our E4 ablation (Table 3), we found that the specific initialization value matters less than having per-layer control: bias = 0 and bias = -2 converge to similar final performance.

A.7 Sparse routing: skipping inactive adapters

When multiple adapters are loaded, most have near-zero gate activation for any given input. We implemented top- k routing that skips adapters below an activation threshold. Pilot result: top-3 routing achieved 26.2 tokens/s vs. 14.8 tokens/s for evaluating all adapters, a 77% throughput increase with no measurable PPL degradation.

A.8 PEFT LoRA vs. custom Expert class

Our Expert class injects a low-rank perturbation into the MLP’s gate and up projections before the nonlinearity, while standard PEFT LoRA adds the perturbation after each linear layer. In pilots, our Expert class showed larger general PPL degradation without gating ($+120\%$ vs. $+25\text{--}32\%$ for PEFT). With gating, both approaches converge to similar final

quality (Table 1). We use our Expert class because it enables clean delta computation: $\text{adapter}(x) - \text{base}(x)$ can be computed in a single forward pass per layer.

B Inference Efficiency: Architecture-Enabled Optimizations

The Grove architecture enables a family of inference optimizations that are *intrinsically tied* to its core components—the per-layer delta gate, hot-pluggable adapters, and bridge surrogates. Several of these optimizations are unavailable to standard dense models or conventional LoRA deployments. We document them here as architectural USPs, noting which have been validated and which remain predictions.

B.1 Gate-Informed Layer Skipping

Mechanism. Layers where the gate activation is consistently near zero for the target domain contribute negligibly to the output. These layers can be skipped entirely—pure residual passthrough, zero compute, zero memory reads.

Unique to Grove? Partially. Mixture-of-Depths [?] and ShortGPT [?] skip layers in dense models, but require either training-time routing or static Block Influence analysis. Our gate provides a *domain-conditioned, pre-trained* skip signal at zero additional cost: the gate was trained for knowledge routing and the skip decision is a free byproduct.

Validated. Skipping 20 of 36 layers yields 228 tok/s ($5.6\times$ HuggingFace baseline) with zero domain PPL loss when the gate conditionally routes general text through the full base model.

B.2 Bridge Surrogates (Zebra Layers)

Mechanism. Replace a full transformer block ($\sim 436\text{M}$ FLOPs, ~ 470 MB weight reads) with a lightweight LoRA bridge ($\sim 0.5\text{M}$ FLOPs, ~ 2 MB reads)—a 99.6% reduction in both compute and memory bandwidth. The bridge is trained via distillation or end-to-end with the language modeling loss to approximate the skipped block’s residual contribution.

Unique to Grove? Yes. The bridge can be packaged *inside* the expert: each domain expert includes not only adapter weights and gate parameters but also bridge surrogates for layers that are redundant *for that domain*. Different experts may bridge different layers, creating domain-specific “zebra” patterns of active and surrogated blocks.

Validated. A rank-64 bridge on block L20 recovers quality to within $+0.6\%$ domain PPL of the full block, at 0.1% of its compute cost. Replicated across 3 seeds.

Prediction. Bridging 4–6 additional layers beyond the 20 skipped would bring the total “surrogated” fraction to 67–72% of all layers, adding only ~ 0.015 ms overhead per token.

B.3 Differential Quantization per Projection

Mechanism. Not all weight matrices within a transformer block are equally sensitive to quantization. We measured per-projection FP4 sensitivity on Qwen3-8B and found:

- **Robust** (safe for FP4): `q_proj` (-0.1%), `k_proj` (-0.7% , *improves*), `down_proj` (-0.1%)—together $\sim 50\%$ of parameters.
- **Sensitive** (keep at FP8): `gate_proj` ($+3.6\%$), `up_proj` ($+1.8\%$), `v_proj` ($+1.2\%$), `o_proj` ($+0.3\%$).

Quantizing only the robust projections to FP4 while keeping sensitive ones at FP8 yields $\sim 25\%$ bandwidth savings with net PPL *improvement*.

Unique to Grove? The measurement itself is not, but the *interaction with the gate* is. The gate modulates how much quantization error propagates to the output: at $\text{gate} = 0.1$, a 5% quantization error in the adapter path contributes only 0.5% to the final output. This means low-gate layers tolerate more aggressive quantization of both base weights and adapter weights. No published quantization scheme exploits this gate-mediated error dampening.

Validated. Per-projection sensitivity measured on C4 validation (Section above). Combined scheme not yet end-to-end benchmarked.

B.4 Gate as Precision-Budget Allocator

Mechanism. A naive approach would pack weights as multiple INT4 registers (e.g., four INT4 values recovering INT16). However, integer formats lack the dynamic range needed for gradient computation during training. Since the grove uses the same memory layout for both training and inference, we propose *segmented mantissa* storage: each weight retains its sign and exponent at full precision, while the mantissa is split into segments that can be read incrementally.

- **Sign + exponent** (6 bits): always read. Preserves the full dynamic range of floating point at every precision level.
- **Mantissa segment A** (2 bits): coarse magnitude. Combined with sign+exponent, yields a minimal FP8-like representation.
- **Mantissa segments B, C** (4 bits each): successive refinements. All segments together recover the full 10-bit mantissa (FP16 precision).

The gate controls how many mantissa segments are read per layer:

$$W_{\text{eff}} = \begin{cases} \text{FP}(s, e, m_A) & \text{if } \sigma(g) < \tau_1 \text{ (8 bits, } 1/2 \text{ bandwidth)} \\ \text{FP}(s, e, m_A, m_B) & \text{if } \tau_1 \leq \sigma(g) < \tau_2 \text{ (12 bits)} \\ \text{FP}(s, e, m_A, m_B, m_C) & \text{if } \sigma(g) \geq \tau_2 \text{ (16 bits, full precision)} \end{cases} \quad (4)$$

Memory footprint is constant (16 bits per weight). Memory *bandwidth* scales with the gate: low-gate layers read 8 bits per weight (sign + exponent + segment A), high-gate layers read all 16 bits. On bandwidth-bound hardware, this halves the cost of non-critical layers while maintaining floating-point semantics at every precision level—critical for gradient computation during training.

Why segmented mantissa, not packed integers? Training requires floating-point gradients. Integer-packed formats (e.g., four INT4 registers) would need dequantization before any gradient computation, destroying the shared code path between training and inference. With segmented mantissa, every precision level is natively floating-point: the sign and exponent are always present, only the mantissa resolution varies. The backward pass can use the same variable-precision weights as the forward pass without format conversion.

Combined with bridges. For layers where the gate is near zero, the full transformer block can be replaced by a bridge surrogate (rank-64 LoRA, 0.1% of compute). Unlike layer skipping—which we showed causes compounding error in autoregressive generation (all 19 tested layers diverge within 2–19 tokens)—bridges produce real output and avoid error propagation. The gate controls both precision *and* compute budget:

Gate	Precision	Compute	Bandwidth
≈ 0	FP8 ($s+e+m_A$)	Bridge (0.5M FLOPs)	8 bits/weight
≈ 0.5	FP12 ($s+e+m_{AB}$)	Full layer (300M FLOPs)	12 bits/weight
≈ 1	FP16 ($s+e+m_{ABC}$)	Full layer + adapter	16 bits/weight

Unique to Grove? Yes. The gate simultaneously controls four resources with one learned signal: (1) expert routing, (2) forward precision, (3) compute budget (full vs bridge), (4) backward precision for training. Standard quantized models use uniform precision. MoE models route but do not modulate precision or compute. Only the gated architecture enables per-layer, per-token, dynamic resource allocation.

Training and inference share the same code path. The base model weights are stored in segmented FP16 format. No mode switch between training and inference. Both the forward and backward pass use the same variable-precision read controlled by the gate. LoRA adapter parameters are always in BF16. The base model weights never change; only the adapter is updated.

Not yet validated. The segmented mantissa format requires custom CUDA kernels for variable-width memory reads. INT4 requires per-group quantization (groups of 128) for viable quality. Bridges validated at +0.6% PPL at 0.1% compute.

Inference performance (validated). With FP8 base weights, pre-computed expert routing, and CUDA graph capture of the full decode step (enabled by `flash_attn_with_kvcache` with tensor-based `cache_seqlens`), the grove achieves 81.6 tok/s base and 69.6 tok/s with an active expert on a single RTX PRO 6000 Blackwell (75% and 64% of the 109 tok/s theoretical bandwidth limit). Expert adapter overhead is 14.7%, dominated by the LoRA matmul compute (<1% of FLOPs) rather than kernel launch overhead (eliminated by CUDA graphs).

Quantization damages the lens, not the crystal. Mean PPL masks a critical effect: INT4 quantization increases the 95th-percentile per-token loss by $4\times$ while mean PPL rises only 3–5%. The worst-case tokens—precisely the decision points where coherent generation depends on accurate next-token probabilities—are disproportionately affected. Multiple-choice accuracy (factual recall) remains 100% at INT4, confirming that *declarative knowledge* (crystal) survives quantization while *generative coherence* (lens) degrades. This distinction is invisible to standard PPL reporting.

Adapters restore the lens on quantized models. A rank-16 DeltaGated adapter trained on an INT4 base model reduces the tail-loss ratio (P95/mean) from $1026\times$ to $6.1\times$ —compared to $4.4\times$ on the FP16 base. Gate selectivity is identical regardless of base model precision (measured at -0.015 on both FP16 and INT4), indicating that the gate learns the same routing pattern on either substrate. The base model can therefore be aggressively quantized; the adapters restore the coherence that quantization removed. This has a practical implication: an INT4 base model (4 GB) with 100 adapters (~ 2.8 GB) gives a 100-domain system in under 7 GB of VRAM.

B.5 Gate-Informed Speculative Decoding

Problem. The variable-precision and bridge mechanisms (Sections B.4–B.2) provide significant speedups, but at a small PPL cost that compounds over long sequences. After N tokens of autoregressive generation, accumulated drift can degrade coherence—a fundamental limitation of any approximate inference path.

Mechanism. We propose using the grove’s own architecture for speculative decoding [28, 29]:

- **Draft path:** FP8 (mantissa segment A only) with bridge surrogates for low-gate layers ($\sigma(g) < \tau$). This is the fast, approximate path already described above.
- **Verification path:** Full FP16 precision (all mantissa segments) with real transformer blocks and active adapters. This produces the exact target distribution.

The draft generates k candidate tokens; the verifier scores all k in a single parallel forward pass using standard rejection sampling [28] to guarantee the output distribution is identical to the full-precision model.

Why the grove architecture is uniquely suited:

1. **No extra model.** Standard speculative decoding requires a separate draft model in memory. Here, draft and verifier are the same weights read at different mantissa resolution—zero additional VRAM.
2. **Gate as acceptance oracle.** The gate predicts where the draft will fail: low gate $\rightarrow$ base model suffices $\rightarrow$ high acceptance rate. High gate $\rightarrow$ adapter needed $\rightarrow$ likely rejection. This enables adaptive speculation window sizing: speculate more aggressively when gates are low, verify sooner when gates rise.
3. **Per-layer selective verification.** Unlike standard speculative decoding which runs the entire verifier, only layers where $\sigma(g) \geq \tau$ need full-precision verification. Low-gate layers remain in INT4 even during the verification pass.
4. **Bridges and $2\times$ bandwidth ratio amplify the speedup.** The draft reads sign + exponent + mantissa segment A (8 bits/weight); the verifier reads all segments (16 bits/weight)—a $2\times$ bandwidth reduction before considering bridges. With bridges

replacing 20 of 36 layers at 0.1% compute, the draft path is estimated at $3\text{--}5\times$ faster than the verification path.

5. **Compounding error is eliminated.** Rejection sampling resets the distribution to the exact target at every rejection point. Drift never accumulates beyond the speculation window of k tokens.

Not yet validated. This mechanism requires implementing the rejection sampling loop and measuring end-to-end throughput including acceptance rates. The key empirical question is whether the acceptance rate on low-gate tokens is high enough ($> 80\%$) to justify the speculation overhead.